Brasil Participativo 2.0 as a Decidim SaaS: An Ecosystem of Fork-Free Modules

Abstract

Brasil Participativo, the Brazilian federal government's participation platform, runs as a multi-tenant SaaS on Decidim 0.32.x. One deployment serves many organizations, each isolated in its own PostgreSQL schema, behind federated national identity. This article describes the ecosystem of fifteen custom and third-party modules that makes this possible without forking Decidim core. It presents the main components: schema-per-tenant multi-tenancy, GovBR federated authentication and government theming, a two-level Organ/Sector hierarchy that delegates process administration to public servants without granting them platform-wide privileges, per-step governance overrides for meetings, proposals and comments, and structured content modules for categorization, questionnaires and homepage composition. Four engineering patterns keep the platform mergeable with upstream: one engine per component, behavior added by prepended concerns, view overrides by Deface or path mirroring, and content-block manifest mutation. A documentation convention turns the module set into a backlog of upstream candidates while feature proposals are closed on MetaDecidim. The ecosystem is open source, and each generalizable feature is marked as an upstream candidate.

1 Introduction

Decidim [1] is designed as a single-organization platform: one deployment, one organization, one community. Running it as software as a service for a federal government, with many organizations on one deployment, national identity and public-servant administrators under civil-service constraints, requires capabilities that the core does not ship. The obvious path is a fork.

Brasil Participativo 2.0 [2] took another path, building on previous work by the Decidim community, particularly Octree, whose contributions provided reusable components for the new architecture. Building on this collaborative ecosystem, Brasil Participativo adopted a fork-free architecture in which every customization is implemented as a self-contained engine or gem, no core file is edited, and the platform tracks upstream Decidim 0.32.x directly.

Here we describe the rest of the platform: the multi-tenancy layer that turns one deployment into a SaaS, the identity and theming layer that makes it a government platform, the administration layer that mirrors the hierarchy of the public service, and the participatory-space overrides that adapt Decidim's components to federal consultation practice. We also describe the engineering patterns that keep the whole mergeable with upstream.

2 Statement of need

Extending Brasil Participativo as a shared digital participation service for Brazilian municipalities requires addressing four interconnected challenges, each corresponding to a layer of the architecture described below.

Many municipalities, one infrastructure. Brazilian municipalities vary significantly in their technical, financial, and institutional capacities. While larger cities may maintain dedicated digital participation infrastructure, smaller municipalities often lack the resources and technical teams needed to deploy and maintain their own systems. A shared SaaS infrastructure reduces these barriers, allowing multiple municipalities to operate independently, each with its own data, participatory processes, branding, and administrators, while sharing maintenance, hosting, and software updates.

Accessible identity, local autonomy. Citizens should be able to participate without creating new accounts for each municipality. Integration with GovBR, Brazil's national digital identity system, provides a familiar authentication mechanism across local governments. At the same time, each municipality needs its own institutional identity, visual configuration, and administrative autonomy within the shared infrastructure.

Administration reflects municipal government structures. Municipal participation initiatives involve different secretariats, departments, and administrative teams. A centralized administration model creates bottlenecks and requires granting excessive permissions to staff responsible for individual processes. Municipalities need to delegate management responsibilities across their organizational structures, allowing public servants to coordinate specific consultations without receiving platform-wide administrative privileges.

Participation adapts to local governance needs. Municipal participatory processes range from consultations on local policies and urban planning to participatory budgeting and community engagement initiatives. These processes require flexible rules across different phases, including when citizens can submit proposals, organize meetings, or comment on contributions. They also require transparent mechanisms for official government responses, categorization, and data exports, supporting accountability and the continuity of participation.

Meeting these requirements through a shared service also demands a sustainable software architecture. Municipalities should benefit from continuous improvements without depending on individual custom deployments or maintaining separate forks. The challenge is therefore to combine local autonomy and methodological flexibility with shared infrastructure, standardized components, and compatibility with upstream Decidim, reducing the technical and institutional barriers to adopting digital participation at the municipal level.

3 Platform architecture

3.1 Multi-tenancy: a schema per organization

The SaaS property comes from decidim-apartment, the integration gem (built on ros-apartment 3.4) that isolates each organization in its own PostgreSQL schema [4]. The host application currently pins this gem to its v0.1.0-rc branch. The single deployment becomes a service: an organization is a tenant with its own schema, administrators and data, physically separated at the database level while sharing one application codebase, one job infrastructure and one upgrade cycle. Every other module described here depends on this layer.

3.2 Identity and theming: GovBR and decidim-govbr

Authentication is federated through the national GovBR login via a dedicated OmniAuth OAuth2 strategy, omniauth-govbr [5]. The strategy is an unofficial, MIT-licensed gem (version 0.1.0) that uses the authorization-code flow with PKCE, requests the scopes openid email profile govbr_confiabilidades, and returns the user's CPF (the Brazilian taxpayer number) with

the profile data. In the host application, the provider is configured per organization: a setup hook looks up the organization from the request host and loads that organization's client credentials, site URL, redirect URL and scope, so each tenant can have its own GovBR registration. The in-repository decidim-govbr engine holds everything specific to operating inside the Brazilian government ecosystem: GovBR user logic, per-organization color theming, homepage content blocks (including a blog/news cell), and administrative extensions for participatory processes (form validations, metadata, automatic step activation with duplicate handling), plus organization-creation extensions for the system panel. The identity layer is also where the platform's ephemeral-participation feature connects: citizens can act without a full registration, with identity established at the moment of the act, as the chatbot's vote path does. The GovBR authorization handler is registered as an ephemeral verification workflow for that purpose.

3.3 Delegated administration: government_spaces

The most institutionally distinctive module is decidim-government_spaces. It scopes participatory-process administration to a two-level government hierarchy, Organ → Sector. Organization administrators build the hierarchy and invite scoped administrators. A scoped administrator manages only the processes reachable through their memberships and can do everything a native process admin can do except delete. Scoped admins never have admin: true, and access is computed from memberships at request time, so revoking a membership revokes access immediately and no cached roles need to be cleaned up. The module is additive: until organs and sectors exist it does nothing, and the platform behaves like stock Decidim.

3.4 Per-step governance: the bp_* overrides

Three override gems adapt Decidim's most participatory components to federal consultation rules. Each follows the same one-file-per-feature convention (Section 4).

decidim-bp_meetings adds step-scoped control of participant event creation, comments and attachments. Two details matter in practice. The creation flag moves from global to step-scoped and defaults to false for each step. This is a documented breaking change, because a consultation that opens citizen meeting creation by default invites moderation problems. Step-scoped attachments also include citizen upload on the meeting form, which exists in no core version and is flagged with its upstream issue [6].

decidim-bp_proposals applies step-scoped comment gating to proposals and collaborative drafts alike. Its default is true, so existing organizations see no behavior change. The default is the opposite of the one chosen for meeting creation, for the same reason: comments are the norm in consultation, and creation is the exception.

decidim-bp_comments reshapes the comment section itself. Citizens comment at the base level; only administrative profiles reply, in a two-level tree, and admin replies carry an "official" badge. Regular authors may edit or delete their own comment for five minutes, global administrators can delete any comment, and the report button is restricted to administrators and moderators. The gem is wired through concerns prepended onto the core's permissions, form, cell and deletion paths at boot, with no core file edited.

3.5 Structured content: categories, questionnaires, blocks

decidim-categories replaces the native categorization with a hierarchical, space-scoped system. It has its own model with unlimited hierarchy and soft delete (records keep their

historical relationships), and it injects business rules into proposal and meeting forms: category selection is mandatory, only leaf nodes are selectable, and sibling subcategories are mutually exclusive.

decidim-questionnaires shows the established pattern for building on the core forms engine: concern-based overrides wired in Engine#to_prepare, view changes in Deface overrides or mirrored view paths, and explicit tracking of core renames per Decidim version (0.32 renamed Answer to Response across the questionnaire surface). Future form modules are expected to follow it.

The content-block modules extend what organizations can compose on their homepages. decidim-extra_home_blocks mutates already-registered core block manifests to add settings, for instance an inline step timeline in place of the core's modal button. decidim-extra_blocks adds layout-based blocks (call to action, hero, fast proposal) that are picked from a gallery, cached and designed mobile-first.

3.6 Assembly and reuse

decidim-community_templates, a fork of the Decidim-ICE module updated to 0.32 [7], lets organizations import participatory-process templates, that is, how a consultation is supposed to be run, distributed as configuration. Two third-party modules from Octree [8] complete the ecosystem. decidim-toggle gives any module a standard surface in the organization settings (the chatbot's configuration uses it). decidim-ephemeral_participation, grafted from Decidim Barcelona, lets citizens participate without registration; it is provisional until an equivalent lands in core.

3.7 Host application and deployment

The modules are assembled by a Decidim host application, participa [9], which is itself open source. It is based on Decidim 0.32.1 (Rails 8.1, Ruby 3.4.7, Node 22.14.0) and declares the modules in its Gemfile: the two in-repository engines (decidim-govbr and decidim-chatbot) by path and the remaining thirteen as git gems, mostly from the lappis-unb/decidimbr group and two from Octree. In total the Gemfile lists fifteen module gems. The host adds the glue that only the integration can supply, such as switching the tenant for Active Storage requests by host name, restricting the schema dump to the public template schema that Apartment clones for each tenant, and excluding every engine's app/overrides directory from Zeitwerk autoloading because those files only declare Deface overrides.

The application is built as a single container image used in every environment and is deployed to Kubernetes with a Helm chart [10]. The chart defines the web (Puma) and Sidekiq workloads, jobs for database creation, schema upgrades, system-admin creation and asset compilation, a PostgreSQL cluster managed by the CloudNativePG operator, Redis for queues and caching, and gateway or ingress configuration with certificates. A wildcard-host option allows tenants to be served as subdomains of one base domain. The host-application repository also carries an older chart for a single-tenant instance (Decidim 0.29.3), whose documentation describes a companion chart for creating organizations in an existing multi-tenant deployment.

4 The four patterns that keep it mergeable

Four engineering patterns recur across the ecosystem, each aimed at one property: staying mergeable with upstream.

- **One engine per component, one file per feature.** Each gem targets a single core component, and every feature lives in features/<name>.rb with a header that explains the reason and links the originating requirement.
- **Behavior by prepend and `config.to_prepare`, never by editing core files.** Permission checks, forms, commands and cells are extended by concerns prepended at boot, so upgrades re-apply the customizations cleanly.
- **Markup by Deface or view-path mirroring.** Small UI injections use Deface overrides. Wholesale view replacements mirror the Decidim path so that ActionView resolves the engine's copy.
- **Content-block manifests are mutated, not re-registered.** The core registry forbids duplicate names, so extensions subclass the registered cell in an after: initializer, which preserves upstream view inheritance.

A documentation convention sits on top of these patterns and matters as much as the code: every feature README states whether the feature is a generalizable upstream candidate or deliberately Brazil-specific. With the MetaDecidim feature-suggestion channel closed, this lets the module set double as a proposal backlog, and each candidate README is in effect a pre-written upstream proposal.

5 Limitations

Schema-per-tenant isolation keeps the design simple but makes operations share fate: an outage of the deployment affects every organization, and schema-level migrations run across the whole fleet. The per-step override gems are deliberately conservative. Their defaults (creation closed, comments open) encode federal consultation practice and will surprise communities with other norms. Several modules are coupled to the 0.32.x line and to Brazilian constructs, namely GovBR login, CPF-based ephemeral identity and the organ/sector administrative vocabulary, and each needs reimplementation effort to be reused elsewhere. Finally, the ecosystem is meant to shrink: the more features merge upstream, the smaller the maintenance surface becomes, and until then the backlog convention carries real weight.

6 Related work

Multi-tenant Decidim is practiced in several ways across the community: separate deployments per organization, row-level scoping and schema-per-tenant, which trade isolation, cost and upgrade mechanics differently. The schema-per-tenant choice here follows the long-established Apartment approach in the Rails ecosystem and prioritizes data isolation between government organizations. Delegated administration resembles role-scoping work in other civic platforms, but the Organ/Sector model is drawn from the structure of the Brazilian federal administration and not from platform concepts. The fork-free customization approach follows the general Decidim extension architecture (engines, concerns, Deface), and the upstream-candidacy convention responds to the current closure of the proposal channel. The WhatsApp and participatory-texts components, covered in companion articles, extend this architecture to messaging channels and normative-document consultation.

7 Availability

The modules are published in the lappis-unb/decidimbr group on GitLab [2]: the customization gems under components-brasil-participativo/, the multi-tenancy gem under infra/participa-gem, the host application as participa [9] and the Helm chart under infra/charts/decidim-helm [10]. The authentication strategy is on GitHub [5] under the MIT license. The host application and the in-repository engines are licensed under AGPL-3.0-or-later. The licenses of the other module repositories, including the two Octree modules, should be stated by the authors. The ecosystem documentation that this article relies on is maintained alongside the code, with links verified against the 0.32.x line.

8 Conclusion

A federal participation platform turned out to be mostly an integration problem. Tenancy, identity, administrative hierarchy and consultation rules are each modest, but together they separate a demo from a service. The ecosystem keeps the platform current by never leaving upstream: one engine per concern, behavior added by prepending, markup by mirroring, features documented with their requirements, and every generalizable idea filed as an upstream candidate for when the community's proposal channel reopens. The SaaS will run in production, the backlog is written, and the fork was never needed.

References

[1] Decidim: free open-source participatory democracy framework. https://decidim.org

[2] Brasil Participativo: module ecosystem on Decidim 0.32.x. https://gitlab.com/lappis-unb/decidimbr

[3] MetaDecidim: community process and proposals. https://meta.decidim.org

[4] participa-gem (decidim-apartment): schema-per-tenant multi-tenancy for Decidim. https://gitlab.com/lappis-unb/decidimbr/infra/participa-gem

[5] omniauth-govbr: unofficial OmniAuth OAuth2 strategy for GovBR (MIT). https://github.com/paulohtfs/omniauth-govbr

[6] Decidim issue #5726: participant attachments in meetings. https://github.com/decidim/decidim/issues/5726

[7] decidim-community_templates (fork of Decidim-ICE, branch core/bump-32). https://gitlab.com/lappis-unb/decidimbr/components-brasil-participativo/decidim-bp_templates

[8] Octree: decidim-toggle and decidim-ephemeral_participation. https://git.octree.ch/decidim/vocacity/decidim-modules/decidim-toggle ; https://git.octree.ch/decidim/vocacity/decidim-modules/decidim-ephemeral_participation

[9] participa: Decidim host application for Brasil Participativo. https://gitlab.com/lappis-unb/decidimbr/participa

[10] decidim-helm: Helm chart for Decidim. https://gitlab.com/lappis-unb/decidimbr/infra/charts/decidim-helm